

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Abderrahmane Mira de Béjaia
Faculté des Sciences Exactes
Département de Recherche Opérationnelle



Mini-projet présenté
dans le cadre de la préparation du Diplôme de Licence
En Mathématiques Appliquées

Introduction à Netwokx et les réseaux de neurones

Présenté par :
MESROUR LOUNIS
MOKHTARI ANIS BADREDINE
OURTILANE MOHAND

Sous la direction de : Mlle.Aoudia

Année Universitaire 2023 – 2024

Remerciements

Nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Tout d'abord, nous remercions chaleureusement notre encadrante, Mlle Aoudia, pour son encadrement précieux, ses conseils avisés et son soutien constant tout au long de cette aventure académique. Ses remarques constructives et son expertise ont été inestimables pour l'avancement de ce travail.

Nous souhaitons également remercier nos professeurs et collègues de l'université pour leurs enseignements enrichissants et leurs encouragements.

Nos remerciements vont également à nos familles et à nos amis pour leur soutien moral et leur compréhension durant cette période intense. Leur patience et leur motivation ont été une source constante de force et d'inspiration.

Enfin, nous remercions tous ceux qui, de près ou de loin, ont contribué à la réalisation de ce projet. Leur soutien et leurs encouragements ont été essentiels à l'aboutissement de ce travail.

Table des matières

Remerciments	I
Liste des figures	IV
<i>Introduction générale</i>	1
I	2
1 La théorie des graphes	3
Introduction	3
1.1 Introduction	4
1.2 Définition des graphes	4
1.3 Quelques types de graphes	5
1.3.1 Graphe non orienté	5
1.3.2 Graphe orienté (diapgraphe)	5
1.3.3 Graphe simple	5
1.3.4 Graphe complet	5
1.3.5 Graphe connexe	6
1.3.6 Graphe biparti	6
1.3.7 Graphe pondéré	6
1.3.8 Graphe complémentaire	7
1.4 Propriétés des graphes	7
1.4.1 Ordre du graphe	7
1.4.2 Taille du graphe	7
1.4.3 Degré d'un sommet	7
1.4.4 Chaîne	7
1.4.5 Chemin	7
1.4.6 Cycle	8
1.4.7 Circuit	8
1.4.8 Sous-graphe	8
1.5 Représentation matricielle d'un graphe	8
1.5.1 Matrice d'adjacence	8
1.5.2 Matrice d'incidence aux arcs	8
1.6 Algorithmes Classiques	9
1.6.1 Parcours de graphe (DFS , BFS)	9
1.6.2 Recherche De Chemins Les Plus Courts	9
Conclusion	10

2	Networkx	11
	Introduction	11
2.1	création de graphes	11
2.2	Visualisation des graphes	12
2.3	Propriétés des graphes	14
2.4	Algorithmes de graphes	16
	Conclusion	19
II		21
3	Réseaux de neurones	22
	Introduction	22
3.1	Définition de réseaux de neurones	23
3.2	Modélisation d'un neurone	23
3.2.1	Le neurone réel (Anatomie) et neurone artificiel (Formel)	23
3.3	Structure et Fonctionnement des neurones	24
3.3.1	modèles de neurone formel	24
3.3.2	Fonction d'activation :	25
3.4	Les différents types de réseaux de neurones artificiels	26
3.4.1	Réseaux de neurones Feed Forward (réseaux de neurones multicouches)	27
3.4.2	Réseaux de neurones récurrents (RNN)	27
3.5	l'apprentissage profond	28
3.6	implementation d'un reseaux de neurons	29
3.6.1	la partie apprentissage et manipulations des donnees	29
3.6.2	La partie de creation du reseau de neurones	29
	Conclusion	29
4	Application	30
	Introduction	30
4.1	Création du Réseau de Neurones	30
4.1.1	Définition du problème	30
4.1.2	Explications	31
4.2	Visualisation du Réseau de Neurones	33
	Conclusion	34
	Conclusion générale	35
	Bibliographie	35

Table des figures

1.1	Les sept ponts de Königsberg	3
1.2	graphe orienté	5
1.3	graphe complet	5
1.4	graphe non connexe	6
1.5	graphe biparti	6
1.6	graphe G et son complémentaire	7
1.7	un graphe et sa matrice adjacence	8
1.8	un graphe et sa matrice d'incidence	9
3.1	Neurone réel	23
3.2	Neurone artificiel	23
3.3	fonctionnement d'un neurone formel.	24
3.4	graphe d'une fonction sigmoïde	26
3.5	le graphe de la fonction ReLU.	26
3.6	réseaux de neurones multicouches	27
3.7	Réseau de neurones récurrents	27

Introduction générale

La théorie des graphes est une discipline à part entière des mathématiques appliquées. Elle trouve divers applications dans des domaines insoupçonnés pas uniquement dans les domaines techniques mais également dans les sciences sociales. Notre projet s'articule autour de la théorie des graphes, de la modélisation des réseaux de neurones, ainsi que la présentation de l'outil informatique networkx.

Notre choix s'est porté sur ce projet car ses domaines sont cruciaux dans divers domaines. La théorie des graphes et son outil numérique Networkx sont utilisés dans divers domaines pour modéliser des relations entre des objets, comme les réseaux de transports, réseaux sociaux ou bien les réseaux de neurones. Ces derniers sont d'excellents outils pour construire des modèles apprenants performants et efficaces. La performance acquise au moyen de ces modèles a donné lieu à de nouveaux cas d'usage, impossibles à imaginer quelques années auparavant. Les réseaux de neurones peuvent être utilisés dans la médecine et la sécurité grâce à leur capacité à traiter des données visuelles (reconnaissance faciale et détection des objets), et dans la finance et l'économie pour la prévision ...etc.

Dans la première partie de notre projet, nous explorons la théorie des graphes. Nous aborderons les concepts de base, et présenterons le module Python Networkx. Grâce à cet outil nous pouvons créer, manipuler et visualiser des graphes, ce qui nous permettra de mettre en pratique les concepts théoriques étudiés.

La deuxième partie est consacrée aux réseaux de neurones, qui constituent une technologie de l'intelligence artificielle et de l'apprentissage automatique. Nous commençons par définir ce qu'est un neurone artificiel et comment il est modélisé. Nous examinons par la suite leur fonctionnement.

On conclue ce projet par la mise en pratique des connaissances acquises dans les chapitres précédents, en présentant un réseau de neurone simple et visualiser sa structure en utilisant la bibliothèque Networkx.

L'objectif de ce projet est de fournir une compréhension de base de la théorie des graphes et des réseaux de neurones, en combinant les aspects théoriques et pratiques. En maîtrisant ces outils, nous serons mieux équipés pour aborder des problèmes complexes et développer des solutions innovantes dans des domaines variés comme la science des données et l'intelligence artificielle.

Cette approche intégrée nous permet de voir comment les mathématiques appliquées peuvent être utilisées pour modéliser, analyser et résoudre des problèmes concrets, renforçant ainsi notre capacité à innover et à contribuer de manière significative à notre domaine d'étude.

Première partie

1

La théorie des graphes

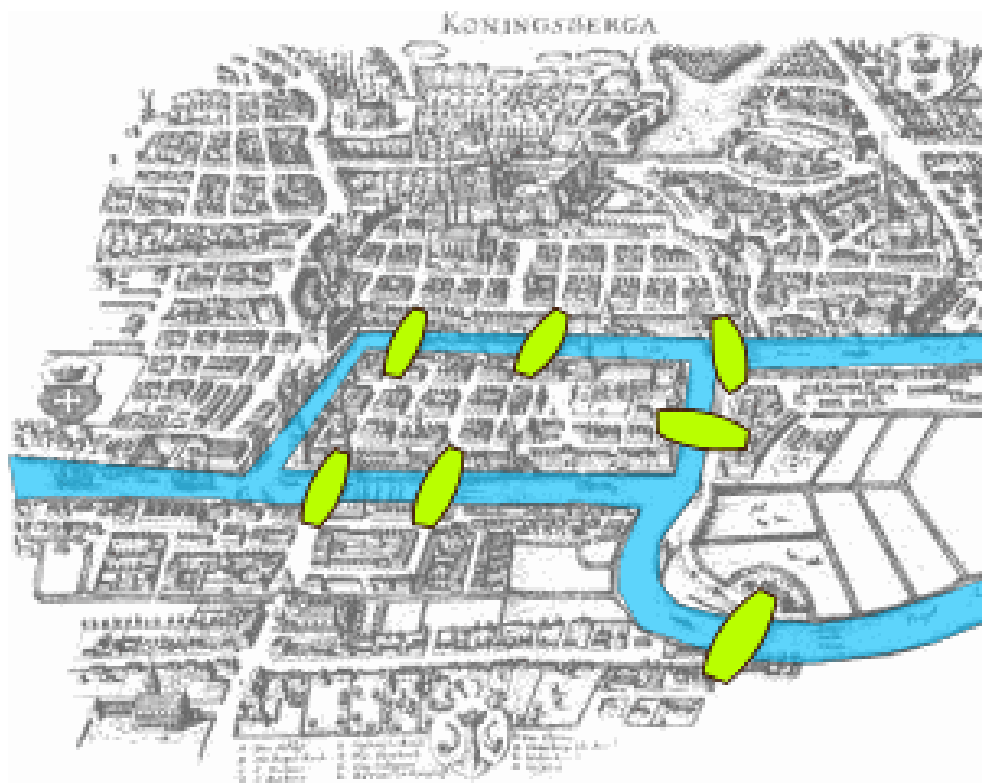


FIGURE 1.1 – Les sept ponts de Königsberg

1.1 Introduction

Avant d'entamer les définitions et les explications on tient à donner un aperçu historique. La théorie des graphes a vu le jour quand le mathématicien suisse Leonhard Euler a essayé de résoudre le problème des sept ponts de Königsberg (Figure 1.1). Le problème des sept ponts de Königsberg cherche à déterminer s'il existe un chemin permettant de revenir à son point de départ en empruntant une seule fois chaque ponts de la ville. En 1735 Leonhard Euler a démontré qu'un tel chemin eulérien n'existe pas pour la configuration de Königsberg, Euler a modélisé le problème en forme de graphe qui représente les ponts avec des arêtes et les villes avec des sommets (ou nœuds). Le travail d'Euler illustre bien l'utilité des graphes pour modéliser les problèmes des réseaux de transport, et d'autres scientifiques après Euler utiliseront les graphes pour résoudre leurs problèmes soit dans l'électricité, les mathématiques, l'informatique, les sciences sociales et d'autres.

Dans ce chapitre, nous étudierons différentes théories, caractéristiques qui nous permettent d'analyser et de classer des graphes, ainsi que des algorithmes qui permettent de résoudre des problèmes liés aux graphes.

Sommaire

Introduction	3
1.1 Introduction	4
1.2 Définition des graphes	4
1.3 Quelques types de graphes	5
1.4 Propriétés des graphes	7
1.5 Représentation matricielle d'un graphe	8
1.6 Algorithmes Classiques	9
Conclusion	10

1.2 Définition des graphes

Un graphe est une représentation visuelle des objets réels et des relations (ou liens) qui existe entre eux. Mathématiquement, Un graphe $G=(V,E)$ est défini par l'ensemble fini $V=(v_1, v_2, \dots, v_n)$ dont les éléments sont appelés sommets, et par l'ensemble fini $E=(e_1, e_2, \dots, e_m)$ dont les éléments sont appelés arêtes. Une arête e de l'ensemble E est définie par une paire non ordonnée de sommets, appelés les extrémités de e . Si l'arête e relie les sommets a et b , on dira que ces sommets sont adjacents, ou incidents avec e , ou bien que l'arête e est incidente avec les sommets a et b . [2]

1.3 Quelques types de graphes

1.3.1 Graphe non orienté

Un graphe non orienté est un graphe dont les arêtes ne sont pas orientées et chaque arête (e) est définie comme suit : $e = (x_i; x_j) = x_i x_j$. Une arête de type $x_i x_i$ est appelée une boucle.

1.3.2 Graphe orienté (diagramme)

Un graphe orienté est un graphe dont chaque arc (e) tel que $e = (x_i; x_j) \in E$ possède une extrémité initiale $I(e)=x_i$ et une extrémité terminal $T(e)=x_j$. Si $I(e)=T(e)$ on dit que e est une boucle.

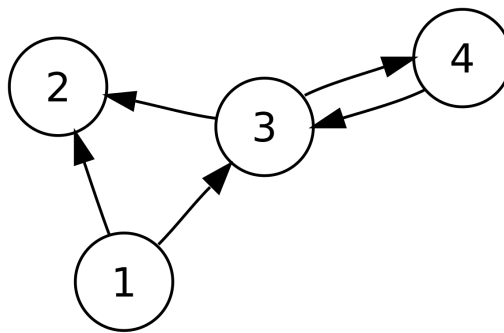


FIGURE 1.2 – graphe orienté

1.3.3 Graphe simple

Est un graphe qui possède une arête au plus entre deux sommets, et ses sommets ne possèdent pas de boucle. Les graphes qui ont ses caractéristiques on les nomme les **multigraphes**.

1.3.4 Graphe complet

Si chaque sommet du graphe est relié directement a tous les autres sommets.

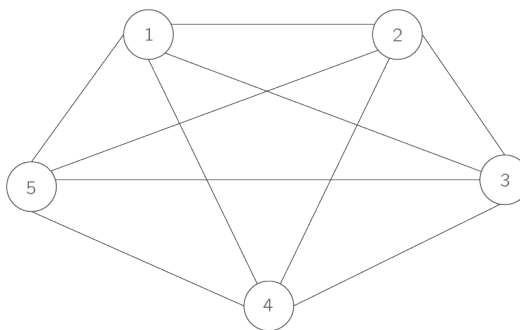


FIGURE 1.3 – graphe complet

1.3.5 Graphe connexe

Un graphe est connexe s'il est possible, à partir de n'importe quel sommet, de rejoindre tous les autres en suivant les arêtes. Un graphe non connexe se décompose en **composantes connexes**. Sur la figure 1.4 on a $V = (1, 2, 3, 4, 5, 6)$ $E = [(1, 3), (1, 4), (2, 3), (3, 4), (5, 6)]$ et les composantes connexes sont $(1, 2, 3, 4)$ et $(5, 6)$.

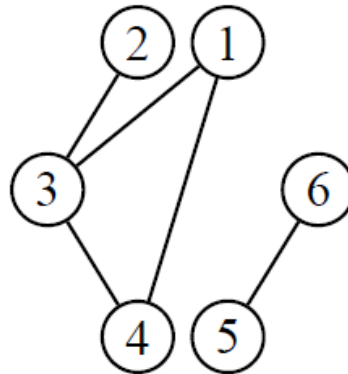


FIGURE 1.4 – graphe non connexe

1.3.6 Graphe biparti

Un graphe est biparti si ses sommets peuvent être divisés en deux ensembles X et Y , de sorte que toutes les arêtes du graphe relient un sommet dans X à un sommet dans Y . Dans la figure 1.4, on a $X = (1, 3, 5)$ et $Y = (2, 4)$.

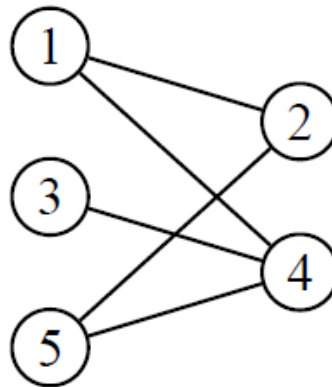


FIGURE 1.5 – graphe biparti

1.3.7 Graphe pondéré

Chaque ar est associée à un poids, que ce soit une distance, un cot, une taille.

1.3.8 Graphe complémentaire

Le graphe complémentaire d'un graphe simple $G = (V, E)$ noté $\bar{G} = (V, \bar{E})$ est un graphe ayant le même ensemble de sommets que G et $\bar{E}$ a exactement les paires de Sommets qui ne sont pas des arêtes de G .

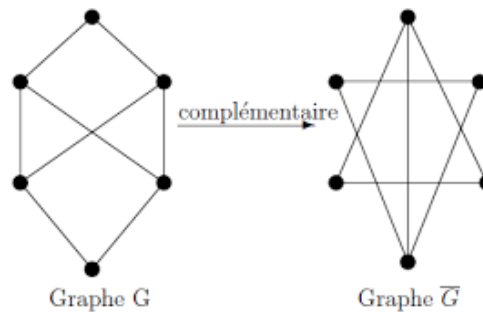


FIGURE 1.6 – graphe G et son complémentaire

1.4 Propriétés des graphes

1.4.1 Ordre du graphe

L'ordre d'un graphe est le nombre de nœuds qu'il contient.

1.4.2 Taille du graphe

La taille d'un graphe est le nombre d'arêtes qu'il contient.

1.4.3 Degré d'un sommet

Le degré d'un sommet est le nombre d'arêtes incidentes à ce sommet.

1.4.4 Chaîne

Une chaîne est une suite finie de sommets reliés deux à deux par une arête.

- Une chaîne Simple est une chaîne qui n'utilise pas deux fois la même arête.
- Une chaîne élémentaire est une chaîne qui n'utilise pas deux fois le même sommet.

1.4.5 Chemin

Dans le graphe G , un chemin menant x_0 à x_k est une suite de sommets reliés successivement par des arcs orientés dans le même sens, que l'on note $(x_0, x_1, \dots, x_k)$.

- Un chemin simple est un chemin qui passe une seule fois par ses arcs.
- Un chemin élémentaire est un chemin qui passe une et une seule fois par ses sommets.

1.4.6 Cycle

Un cycle est une chaîne dont les extrémités coïncident.

1.4.7 Circuit

Un circuit est un chemin dont les extrémités sont confondues.

1.4.8 Sous-graphe

Un sous-graphe d'un graphe G est un graphe dont tous les sommets et toutes les arêtes appartiennent à G.

1.5 Représentation matricielle d'un graphe

1.5.1 Matrice d'adjacence

La matrice d'adjacence est une matrice (n*n) tel que chaque ligne et chaque colonne correspond à un sommet de tel sorte : [8]

$$a_{ij} = \begin{cases} 1 & \text{s'il existe un arc } (x_i, x_j) \text{ ou } x_i = I(e) \text{ et } x_j = T(e), \\ 0 & \text{sinon} \end{cases}$$

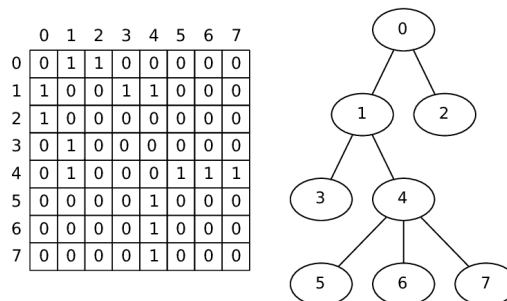


FIGURE 1.7 – un graphe et sa matrice adjacence

1.5.2 Matrice d'incidence aux arcs

La matrice d'incidence aux arcs d'un graphe sans boucles est une matrice (n*m) tel que chaque ligne correspond à un sommet et chaque colonne à un arc et qui prennent comme valeurs.[8]

$$a_{ij} = \begin{cases} 1 & \text{si } x_i \text{ est l'extrémité initiale de } e_j, \\ -1 & \text{si } x_i \text{ est l'extrémité terminale de } e_j, \\ 0 & \text{sinon} \end{cases}$$

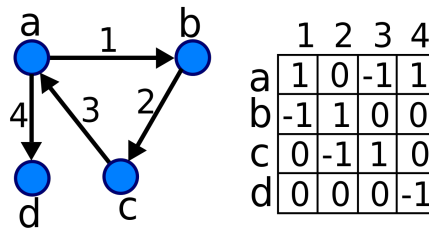


FIGURE 1.8 – un graphe et sa matrice d’incidence

1.6 Algorithmes Classiques

Il existe plusieurs algorithmes classiques pour résoudre différents problèmes liés aux graphes. voici quelques-uns des algorithmes classiques les plus importants en théorie des graphes :

1.6.1 Parcours de graphe (DFS , BFS)

Le parcours de graphes est une opération qui consiste à visiter tous les sommets d’un graphe de manière systématique. Il existe deux algorithmes classiques pour le parcours de graphes : le parcours en profondeur (DFS) et le parcours en largeur (BFS).

Parcours en profondeur (Depth-First Search - DFS)

L’algorithme de recherche en profondeur est utilisé pour parcourir tous les nœuds d’un graphe en suivant les arêtes en profondeur. Il explore les nœuds autant que possible avant de faire marche arrière. Cet algorithme utilisé pour détecter les cycles dans un graphe et pour trouver un chemin entre deux nœuds.[3]

Parcours en Largeur (Breadth-First Search - BFS)

L’algorithme de recherche en largeur est utilisé pour parcourir tous les nœuds d’un graphe en utilisant une exploration en largeur. Il commence par un nœud de départ, puis explore tous ses voisins avant de passer aux voisins des voisins. Cet algorithme peut être utilisé pour trouver le plus court chemin entre deux nœuds d’un graphe non pondéré.[3]

1.6.2 Recherche De Chemins Les Plus Courts

La recherche des chemins les plus courts est un problème fondamental en théorie des graphes, qui consiste à trouver le chemin de coût minimal (poids, distance, temps, etc.) entre deux nœuds dans un graphe. Il existe plusieurs algorithmes classiques pour résoudre ce problème, selon le type de graphe.[3] Voici quels algorithmes :

- **Algorithme de Dijkstra** : Cet algorithme résout le problème du plus court chemin dans un graphe pondéré, où tous les poids des arêtes sont positifs.
- **Algorithme de Bellman-Ford** : Cet algorithme résout le problème du plus court chemin dans un graphe pondéré, même en présence d’arêtes de poids négatifs (mais pas de cycles de poids négatifs).

- **Algorithme de Floyd-Warshall** : Cet algorithme résout le problème des plus courts chemins entre toutes les paires de nœuds dans un graphe pondéré.

Ces algorithmes ont de nombreuses applications dans les systèmes de navigation, les réseaux de télécommunications, l'analyse des réseaux sociaux, la logistique, etc. Ils permettent de déterminer les itinéraires les plus rapides ou les plus économiques dans des graphes représentant des réseaux routiers, des réseaux informatiques ou tout autre type de réseau.

Conclusion

La théorie des graphes offre des outils essentiels pour modéliser et analyser des structures complexes, tels que les réseaux sociaux, réseaux informatiques et d'autres réseaux en biologie. Dans ce chapitre, nous avons couvert les concepts de base, les propriétés des graphes et les algorithmes classiques, posant ainsi les bases théoriques nécessaires pour aborder les problèmes pratiques. Même si cette théorie propose un cadre mathématique strict, elle présente certaines limites pour l'étude des réseaux complexes du monde en pratique.

C'est la raison pour laquelle dans le chapitre suivant, nous allons aborder NetworkX, une bibliothèque Python, qui nous permet de manipuler et analyser des graphes de manière efficace, facilitant ainsi l'étude des structures complexes et la résolution de problèmes réels.

2

Networkx

Introduction

Après avoir établi les fondements théoriques de la théorie des graphes dans le premier chapitre, nous entrons maintenant dans le monde pratique de la manipulation et de l'analyse des graphes à l'aide de la bibliothèque Networkx. Cette dernière est une puissante bibliothèque Python qui offre une multitude d'outils pour créer, manipuler et analyser des graphes complexes de manière efficace.

Dans ce chapitre, nous explorerons les fonctionnalités de NetworkX, en commençant par la création de graphes, la visualisation, l'analyse de graphes et l'application d'algorithmes pour résoudre des problèmes spécifiques.

Sommaire

Introduction	11
2.1 création de graphes	11
2.2 Visualisation des graphes	12
2.3 Propriétés des graphes	14
2.4 Algorithmes de graphes	16
Conclusion	19

2.1 création de graphes

Avec Networkx, on peut créer un graphe en utilisant une des classes suivantes :

- . nx.Graph : Pour les graphes non orientés.
- . nx.DiGraph : Pour les graphes orientés.
- . nx.MultiGraph : Qui est une classe plus flexible autorisant plusieurs liens entre différents noeuds.

. `nx.MultiDiGraph` : Qui est la version orientée de `nx.MultiGraph`.

Il existe plusieurs façons de construire un graphe. On peut le construire en ajoutant les sommets et arêtes.

Exemple :

```
import networkx as nx
# On crée un graphe vide
G = nx.Graph()
# On ajoute les sommets
G.add_node(1) # un sommet
G.add_nodes_from([2, 3]) # une liste de sommets
G.add_node(2, poids=0.5) #Attributs par défaut pour le sommet
# On ajoute les arêtes
G.add_edge(1, 2) # une arête
G.add_edges_from([(1, 3), (2, 1)]) # une liste d'arêtes
G.edges[1, 2]['weight'] = 0.5 #Ajouter un attribut à une arête
```

remarque :si on crée des arêtes directement sans créer des sommets les sommets se formeront automatiquement.

on peut aussi créer un graphe en utilisant la matrice d'adjacence en utilisant la librairie numpy.

Exemple :

```
M=np.array([[0,1,0],[1,1,0],[1,0,0]]) #création d'une matrice avec la fonction array de numpy
G=nx.graphe(M) #transformation de la matrice M en un graphe G
```

2.2 Visualisation des graphes

NetworkX simplifie la visualisation des graphes grâce à son intégration avec Matplotlib. on peut dessiner et visualiser un graphe en appelant certaines commandes de python :

```
import matplotlib.pyplot as plt
nx.draw()
plt.show()
```

la commande **"import matplotlib.pyplot as nx"** est utilisée pour importer le module pyplot dans la bibliothèque matplotlib, ce module fournit une interface qui nous permet de dessiner des graphes, **"nx.draw()"** est une fonction de networkx qui nous permet de dessiner notre graphe, et **"plt.show()"** est une fonction de matplotlib qui permet de voir le graphe.

NetworkX offre de nombreuses options pour personnaliser l'aspect visuel du graphe. On peut spécifier la couleur, l'épaisseur des arêtes, la taille des nœuds. Par exemple : **nx.draw(G, with_labels=True, node_color='blue', node_size=2000, edge_color='black', width=2)**.

Exemple 1 :un graphe simple

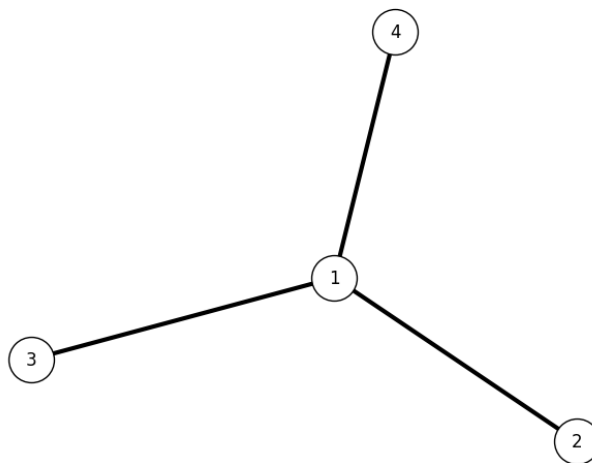
```
import networkx as nx
import matplotlib.pyplot as plt
```

```

G= nx.Graph()
G.add_node(1)
G.add_nodes_from([2,3])
G.add_edge(1,2)
G.add_edges_from([(1,3),(2,1)])
#Si on ajoute une arete avec un sommet inconnu celui-ci sera automatiquement créé
G.add_edge(1,4)
nx.draw(G, with_labels=True , node_size=1200 , node_color='white', edgecolors='black',
width=3)
plt.show()

```

Graphe G :



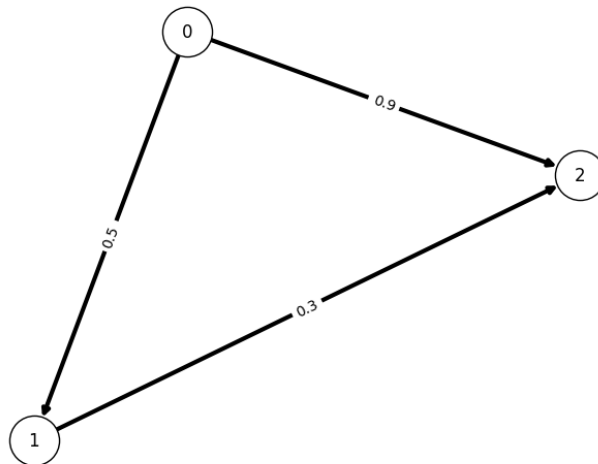
Exemple 2 : création d'un graphe pondéré et orienté

```

import networkx as nx
import matplotlib.pyplot as plt
import numpy as np # Graphe pondéré via la matrice d'adjacence
M = np.array([[0,0.5,0.9],[0,0,0.3],[0,0,0]])
G = nx.DiGraph(M)
pos = nx.spring_layout(G)
nx.draw(G,with_labels=True,pos=pos , node_size=1200 , node_color='white', edgecolors='black',
width=3)
labels=nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G,pos,edge_labels=labels)
plt.show()

```

Graphe G :



2.3 Propriétés des graphes

Il existe de nombreuses fonctions dans Networkx pour récupérer les différentes propriétés d'un graphe. on va citer ces fonction :

- . **'G.number_of_nodes()'** : renvoie le nombre de nœuds dans le graphe .
- . **'G.number_of_edges()'** : renvoie le nombre d'arêtes dans le graphe.
- . **'G.nodes'** : renvoie la liste de nœuds .
- . **'G.edges'** : renvoie la liste de d'arêtes .
- . **'G.degree()'** : renvoie un objet qui répertorie les degrés de tous les nœuds dans le graphe
- .
- . **'list(G.adj[node])'** : renvoie la liste des nœuds voisins du nœud spécifié dans le graphe
- .
- . **'nx.to_numpy_array()'** : Cette fonction convertit le graphe en une matrice d'adjacence
- .
- . **'nx.cycle_basis()'** : pour trouver une base de cycles dans le graphe G. Cette fonction renvoie une liste de cycles, chacun représenté sous forme de liste de nœuds.
- . **'nx.connected_components()'** : pour trouver les composantes connexes dans le graphe
- . Cette fonction renvoie un générateur contenant des ensembles de nœuds, chaque ensemble représentant une composante connexe du graphe.

Exemple :

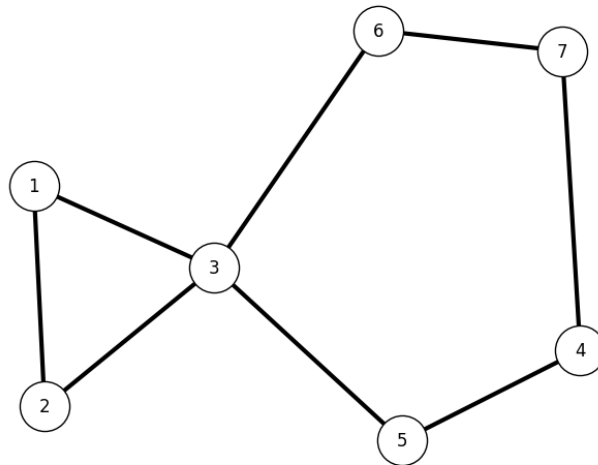
```

import matplotlib.pyplot as plt
import numpy as np
import networkx as nx
G = nx.Graph()
G.add_edges_from([(1, 3), (2, 1), (2, 3), (3,5), (6,7), (6,3),(5, 4),(4,7)])
nx.draw(G,with_labels=True, node_color='white', edgecolors='black' , node_size=1200 ,
width=3)
plt.show()
print("le nombre de nœuds est : \n", G.number_of_nodes())
print("le nombre d'arêtes est : \n", G.number_of_edges())
print("les nœuds sont : \n", G.nodes)
  
```

```

print("les arêtes sont : \n", G.edges)
print("les degres des nœuds du graphe sont : \n", G.degree())
if nx.is_connected(G) :
    print("Le graphe est connexe \n ")
else :
    print("Le graphe n'est pas connexe \n")
    components = nx.connected_components(G)
    components_list = [list(component) for component in components]
    print("Les composantes connexes du graphe G sont :\n", components_list)
adjacency_matrix = nx.to_numpy_array(G)
print("la matrice d'adjacence du graphe G :\n",adjacency_matrix)
cycle_basis = nx.cycle_basis(G)
print("les cycles existant dans le graphe G sont :\n",cycle_basis )
sortie :

```



le nombre de nœuds est :

7

le nombre d'arêtes est :

8

les nœuds sont :

[1, 3, 2, 5, 6, 7, 4]

les arêtes sont :

[(1, 3), (1, 2), (3, 2), (3, 5), (3, 6), (5, 4), (6, 7), (7, 4)]

les degres des nœuds du graphe sont :

[(1, 2), (3, 4), (2, 2), (5, 2), (6, 2), (7, 2), (4, 2)]

la liste des voisins du sommet 2 sont :

[1, 3]

Le graphe est connexe

la matrice d'adjacence du graphe G :

[[0.1.1.0.0.0.0.]

[1.0.1.1.1.0.0.]

[1.1.0.0.0.0.0.]

[0.1.0.0.0.1.]

[0.1.0.0.0.1.0.]

[0.0.0.0.1.0.1.]

[0.0.0.1.0.1.0.]

les cycles existant dans le graphe G sont :

[[5, 3, 6, 7, 4], [1, 2, 3]]

2.4 Algorithmes de graphes

De nombreux algorithmes sont implémentés pour l'étude des graphes dans la bibliothèque Networkx. On présente ici simplement les fonctions de parcours en largeur (et en profondeur) ainsi que la recherche de plus court chemin :

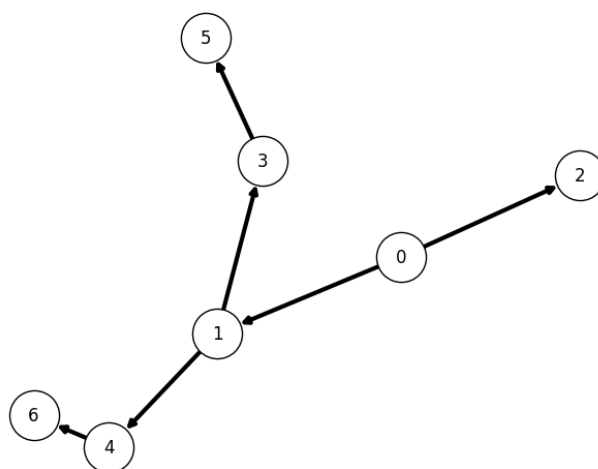
— Les parcours en profondeur (Depth-First Search) :

on utilise la fonction `nx.dfs_tree(G,source)` pour construire un arbre du parcours en profondeur du graphe en commençant par le nœud source et `nx.dfs_edges(G,source)` produit des arêtes dans l'ordre du parcours en profondeur.

Exemple :

```
import networkx as nx
import matplotlib.pyplot as plt
d = {0:[1,2],1:[3,4],3:[5],4:[6]}
G = nx.DiGraph(d)
print("Graphe G : \n")
plt.figure(1)
nx.draw(G,with_labels=True,pos=nx.spring_layout(G),node_size=1200,node_color='white',
edgecolors='black',width=3)
plt.show()
```

Graphe G :



```
parcours_prof = nx.dfs_tree(G, source=1)
```

```

print("Liste du parcours en profondeur : \n")
print(list(parcours_prof.edges()))
print("Arbre du parcours en profondeur : \n")
plt.figure(2)
nx.draw(parcours_prof,with_labels=True,node_size=1200 , node_color='white', ed-
gecolors='black', width=3)

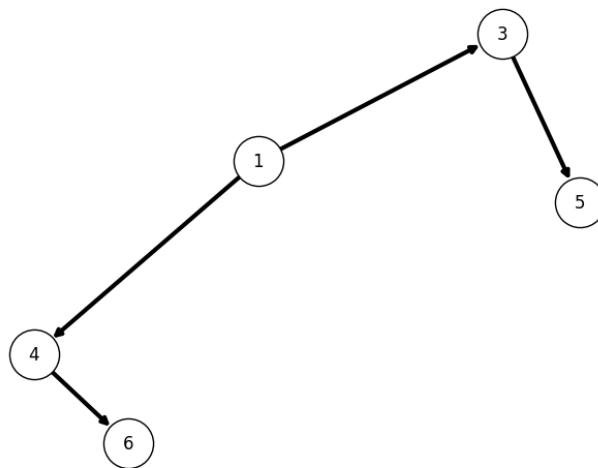
```

Sortie :

Liste du parcours en profondeur :

[(1, 3), (1, 4), (3, 5), (4, 6)]

Arbre du parcours en profondeur :



— **les parcours en largeur (Breadth-First Search en anglais, souvent abrégé BFS) :**

on utilise la fonction `nx.bfs_tree(G,source)` pour construire un arbre du parcour en largeur du graphe en commençant par le nœud source et `nx.bfs_edges(G,source)` produit des arêtes dans l'ordre du parcours en largeur.

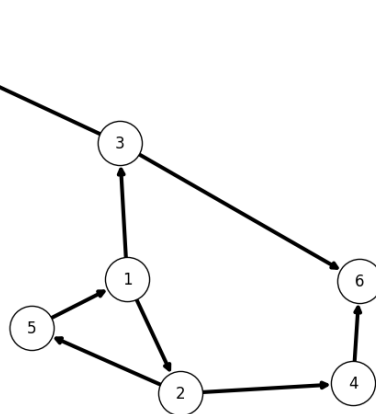
Exemple : application du parcour en largeur sur le graphe G.

```

import matplotlib.pyplot as plt
import numpy as np
import networkx as nx
G = nx.DiGraph(d)
G.add_nodes_from([1, 2, 3, 4, 5, 6, 7]) G.add_edges_from([(1, 2), (1, 3), (2, 4), (2, 5),
(5,1) , (3, 6), (3, 7) , (4,6)]) print("Graphe G : \n")
plt.figure(1)
nx.draw(G,with_labels=True,pos=nx.spring_layout(G),node_size=1200 , node_color='white',
edgcolors='black', width=3)
plt.show()

```

Graphe G :



```

parcours_larg = nx.bfs_tree(G, source=1)
print("Liste du parcours en largeur : \n")
print(list(parcours_larg.edges()))
print("Arbre du parcours en largeur : \n")
plt.figure(2)
nx.draw(parcours_larg,with_labels=True,node_size=1200 , node_color='white', ed-
gecolors='black', width=3 )

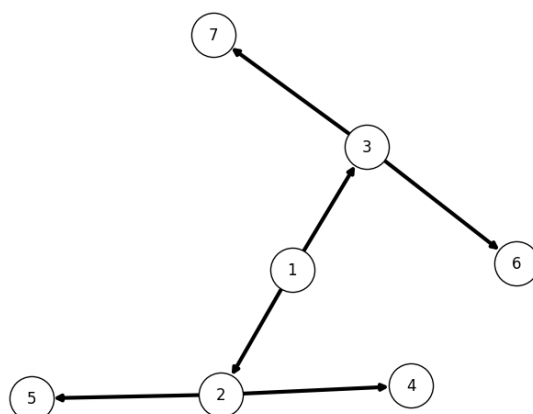
```

Sortie :

Liste du parcours en largeur :

[(1, 2), (1, 3), (2, 4), (2, 5), (3, 6), (3, 7)]

Arbre du parcours en largeur :



— Recherche du chemin le plus court :

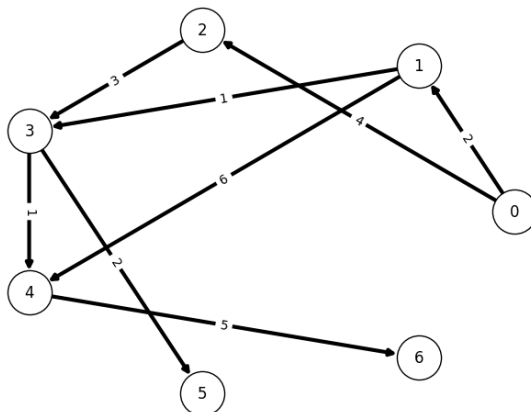
pour rechercher le chemin le plus court entre deux nœuds ,on utilise la fonction **nx.shortest_path**

Exemple :

```
import networkx as nx
import matplotlib.pyplot as plt
d = {0:{1:{'weight':2},2:{'weight':4}},1:{3:{'weight':1},4:{'weight':6}},2:{3:{'weight':3},
5:{'weight':2}},4:{6:{'weight':5}} }
G = nx.DiGraph(d)
print("Graphe G : \n")
pos=nx.circular_layout(G)
nx.draw(G, with_labels=True,pos=pos , node_size=1200 , node_color='white', ed-
gecolors='black', width=3)
labels = nx.get_edge_attributes(G,'weight')
nx.draw_networkx_edge_labels(G,pos,edge_labels=labels)
plt.show()
shortest_path=nx.shortest_path(G, source=0, target=4)
print("Plus court chemin de 0 à 4 : ",shortest_path)
```

Sortie :

Graphe G :



Plus court chemin de 0 à 4 : [0, 1, 4]

Conclusion

Dans ce chapitre, nous avons donné un aperçu sur les fonctionnalités de Networkx, une bibliothèque Python puissante pour la manipulation et l'analyse des graphes. Nous avons commencé par apprendre à créer différents types de graphes et extraire leur propriétés avec Networkx et les visualiser avec matplotlib, ensuite on a utilisé cet outil pour appliquer certains algorithmes de la théorie des graphes.

En conclusion, cette bibliothèque nous a permis de mettre en pratique les concepts de la théorie

des graphes. Dans la prochaine étape, nous entamons la deuxième partie de notre projet, consacrée à l'étude des réseaux de neurones.

Deuxième partie

3

Réseaux de neurones

Introduction

Toutes les nouvelles recherches dans l'informatique sont orientées vers **l'intelligence artificielle**. Et ses recherches visent à rendre les machines plus performantes et plus présentes dans le quotidien de l'Homme, c'est-à-dire rendre les ordinateurs capables d'exécuter des tâches humaines, comme l'assistance de l'humain dans les travaux de la recherche des solutions et l'analyse des problèmes aux seins des entreprises, ou encore plus avancées des intelligences artificielles qui sont capables de simuler les sentiments humains ou des robots qui sont utilisés dans la médecine et l'industrie. Et toutes ces super-machines ont vu le jour grâce à des méthodes mathématiques et technologies informatiques. Ces méthodes consistent à avoir un grand nombre de données à traiter avec des statistiques, ensuite ces données serviront d'exemple pour que la machine apprenne et tout ce processus est le domaine de l'intelligence artificielle appelé **apprentissage automatique** ou en anglais (**machine Learning**), et il existe un sous-domaine de ce dernier, encore plus avancé et utilisé dans toutes les intelligences artificielles les plus compliquées qui est **l'apprentissage profond** ou (**deep Learning**) en anglais. Cette méthode utilise une quantité plus importante de données, et pour que la machine apprenne avec ce grand échantillon de données on utilise les **réseaux de neurones** qui est une méthode qui consiste à mimer le fonctionnement d'un cerveau humain c'est-à-dire que la machine traitera les données comme le cerveau humain le fait, et cette combinaison entre la capacité humaine et la vitesse de calcul d'une machine donne naissance à des ordinateurs et des robots qui peuvent faire des prédictions dans plusieurs domaines, traiter des langues et les comprendre, et rendre la capacité d'apprentissage d'un ordinateur plus performante.

L'intelligence artificielle est un domaine très vaste et une rencontre de beaucoup d'autre domaine de l'informatique et mathématiquement pour cela dans ce chapitre en sera recentrée sur les **réseaux de neurones**.

Sommaire

Introduction	22
3.1 Définition de réseaux de neurones	23
3.2 Modélisation d'un neurone	23
3.3 Structure et Fonctionnement des neurones	24
3.4 Les différents types de réseaux de neurones artificiels	26
3.5 l'apprentissage profond	28
3.6 implementation d'un reseaux de neurons	29
Conclusion	29

3.1 Définition de réseaux de neurones

Les réseaux de neurones sont des modèles informatiques inspirés par le fonctionnement du cerveau humain. Ils sont utilisés pour résoudre des problèmes complexes de classification, de reconnaissance de formes, de prédiction et d'optimisation. Un réseau de neurones est composé de couches de neurones interconnectés, où chaque neurone est une unité de calcul élémentaire. Chaque neurone reçoit des entrées pondérées, les combine à l'aide d'une fonction d'activation, puis transmet le résultat à la couche suivante. L'apprentissage dans les réseaux de neurones est souvent supervisé, où le réseau ajuste ses poids et ses biais pour minimiser une fonction de perte, afin de produire des prédictions précises. Les réseaux de neurones sont largement utilisés dans des domaines tels que la vision par ordinateur, le traitement du langage naturel, la reconnaissance de la parole, la robotique et bien d'autres.[7]

3.2 Modélisation d'un neurone

3.2.1 Le neurone réel (Anatomie) et neurone artificiel (Formel)

Les réseaux de neurones artificiels sont des modèles mathématiques inspirés de la biologie. Le neurone artificiel, est né initialement du désir de modéliser le fonctionnement d'un neurone biologique.

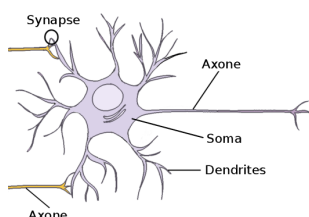


FIGURE 3.1 – Neurone réel

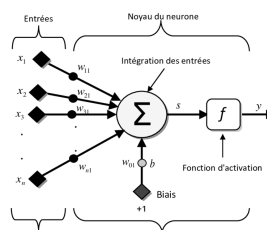


FIGURE 3.2 – Neurone artificiel

Neurone réel (Anatomy)	Neurone artificiel (Formel)
Synapses : Envoyer des impulsions électriques aux neurones voisins.	Poids de la connexion : Facteurs multiplicateurs qui affectent l'influence de chaque entrée sur la sortie du neurone.
Axon : Transfère les signaux d'impulsion électrique du corps cellulaire à la synapse.	Sortie : Directement une des sorties du système ou peut être distribuée vers d'autres neurones.
Soma(cellule) : Le corps cellulaire qui contient la plupart des organites de la cellule.	Neurone : Est une unité de calcul qui reçoit des informations, effectue des calculs simples avec celles-ci et les transmet ensuite.
Noyau : Contient l'ADN de la cellule.	Noyau de neurone : Intègre toutes les entrées et le biais et calcule la sortie du neurone selon une fonction d'activation.
Dendrites : Reçoivent des impulsions électriques des neurones voisins.	Entrées : Directement les entrées du système ou peuvent provenir de d'autres neurones.

TABLE 3.1 – comparaison entre le neurone naturel et formel.[5]

3.3 Structure et Fonctionnement des neurones

3.3.1 modèles de neurone formel

Un neurone artificiel, également appelé perceptron, est l'unité de base dans un réseau de neurones artificiels [9] . Voici comment il fonctionne :

1. **Entrées** : Un neurone artificiel reçoit des signaux d'entrée à partir d'autres neurones ou directement à partir des données d'entrée.
2. **Poids** : Chaque signal d'entrée est multiplié par un poids correspondant. Les poids représentent la force de connexion entre les entrées et le neurone. Ils peuvent être modifiés pendant la phase d'apprentissage pour que le neurone produise les sorties désirées.
3. **Somme pondérée** : Les produits des entrées et de leurs poids correspondants sont sommés ensemble, généralement accompagnés d'un terme de biais. La somme pondérée est calculée comme suit :

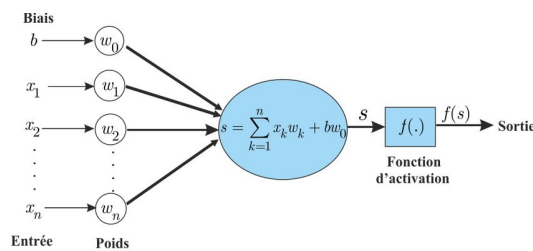


FIGURE 3.3 – fonctionnement d'un neurone formel.

$$S = \sum_{i=1}^n (x_i \cdot w_i) + b$$

Où x_i sont les entrées, w_i sont les poids correspondants, b est le biais et n est le nombre d'entrées.

4. **Fonction d'activation** : La somme pondérée est ensuite introduite dans une fonction d'activation. Cette fonction détermine si le neurone doit "s'allumer" (produire une sortie) en fonction de la valeur de la somme pondérée. Les fonctions d'activation couramment utilisées comprennent la fonction sigmoïde, la fonction tangente hyperbolique, la fonction ReLU (Rectified Linear Unit), etc.
5. **Sortie** : La sortie du neurone est le résultat de la fonction d'activation appliquée à la somme pondérée. C'est la valeur finale produite par le neurone, qui peut être transmise à d'autres neurones dans le réseau.

En résumé, un neurone artificiel combine les entrées pondérées, les soumet à une fonction d'activation, puis produit une sortie en fonction de cette combinaison. Ces neurones sont ensuite organisés en couches pour former des réseaux de neurones artificiels capables d'apprentissage et de résolution de problèmes complexes.

3.3.2 Fonction d'activation :

Il existe plusieurs fonctions d'activation. Nous pouvons citer les types suivants :

- **Fonction sigmoïde** : La fonction sigmoïde est très utilisée dans les réseaux de neurones, strictement Croissante. Un exemple de fonction sigmoïde est définie par l'équation suivante :

$$\frac{1}{1 + e^{-x}}$$

Ou plus généralement

$$\frac{1}{1 + e^{-\lambda x}}$$

avec λ paramètre de la fonction.

La courbe sigmoïde possède pour asymptotes les droites d'équation $y = 0$ et $y = 1$. Elle a pour Centre de symétrie le point I de coordonnée $(0, 1/2)$. Sa courbe est donnée par la figure suivante Cette fonction transforme les valeurs d'entrée en une plage comprise entre 0 et 1. Son principe est de convertir une valeur continue en une probabilité.

- **Fonction seuil** : Est une fonction qui produit une sortie binaire, en fonction d'une valeur seuil fixée. Si la valeur d'entrée dépasse le seuil, la sortie activée (généralement a 1), sinon la sortie désactivée (généralement a 0). Elle est définie comme suit :

$$f(x) = \begin{cases} 1 & \text{si } x > \text{seuil} \\ 0 & \text{sinon} \end{cases}$$

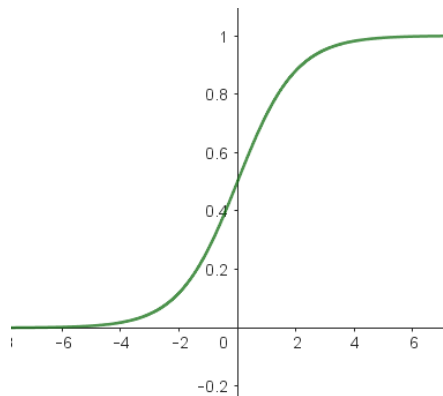


FIGURE 3.4 – graphe d’une fonction sigmoïde

- **Fonction ReLU (Rectified Linear Unit) :** Est l’une des fonctions les plus utilisées. Elle est définie comme suit :

$$f(x) = \max(0, x)$$

ReLU retourne 0 pour toutes les valeurs négatives de x et renvoie simplement x pour toutes les valeurs positives. Cela signifie qu’elle est très efficace pour la propagation du signal dans le réseau, car elle ne souffre du problème de disparition du gradient rencontrée avec les fonctions telles que la sigmoïde. ReLU est également rapide à calculer, ce qui en fait un choix privilégié pour de nombreux réseaux de neurones modernes.

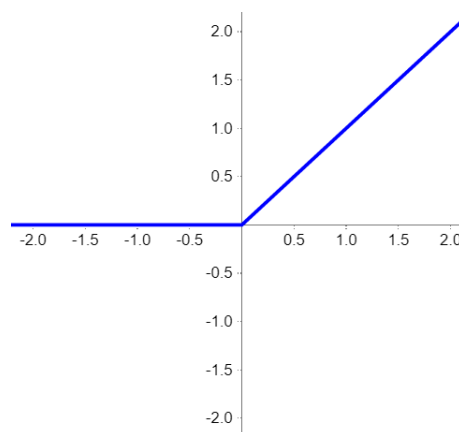


FIGURE 3.5 – le graphe de la fonction ReLU.

3.4 Les différents types de réseaux de neurones artificiels

Il existe plusieurs types de réseaux de neurones, chacun adapté à des tâches spécifiques en fonction de la structure et de l’architecture du réseau. Voici quelques-uns des types les plus courants :

3.4.1 Réseaux de neurones Feed Forward (réseaux de neurones multicouches)

Les réseaux Feed Forward sont la forme la plus simple de réseaux de neurones. Ils se composent de plusieurs couches de neurones, chaque neurone étant connecté à tous les neurones de la couche suivante. Ils sont utilisés pour des tâches comme la classification, la régression et la reconnaissance de motifs.

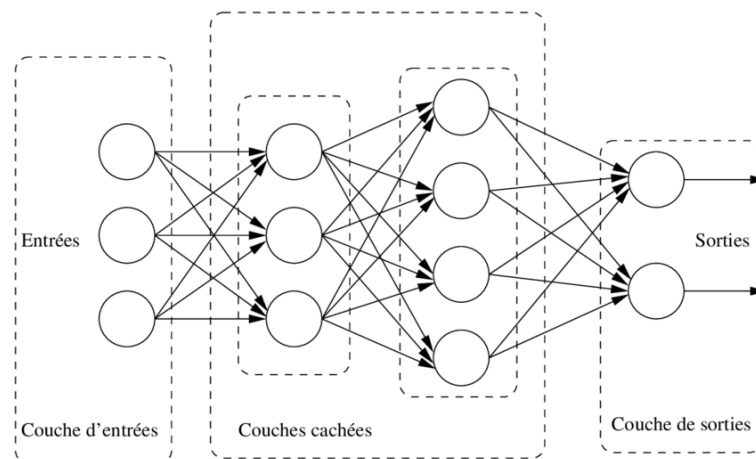


FIGURE 3.6 – réseaux de neurones multicouches

3.4.2 Réseaux de neurones récurrents (RNN)

Ce sont des réseaux de neurones spécialisés qui utilisent le contexte des entrées lors du calcul de la sortie. La sortie dépend des entrées et des sorties calculées précédemment. Ainsi, les RNN conviennent aux applications où les informations historiques sont importantes. Ces réseaux nous aident à prévoir les séries chronologiques dans les applications commerciales et à prévoir les mots dans les applications de type chatbot. Ils peuvent fonctionner avec différentes longueurs d'entrée et de sortie et nécessitent une grande quantité de données.

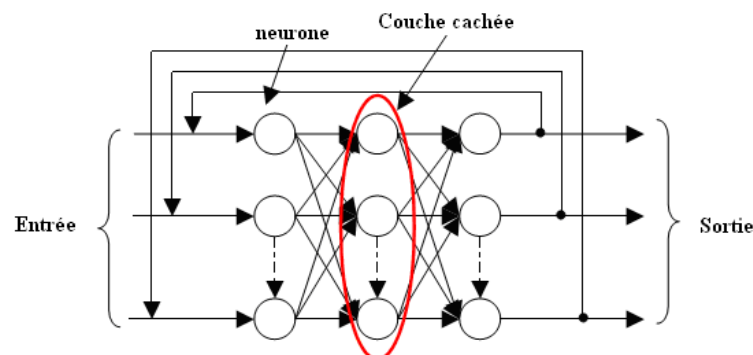


FIGURE 3.7 – Réseau de neurones récurrents

3.5 l'apprentissage profond

L'apprentissage profond fonctionne en utilisant des réseaux de neurones artificiels pour apprendre des représentations hiérarchiques des données [1]. Voici une explication simplifiée de son fonctionnement :

Initialisation : Au début du processus d'apprentissage, les poids des connexions entre les neurones du réseau de neurones sont généralement initialisés aléatoirement.

Propagation avant (forward propagation) : Lorsqu'une donnée est introduite dans le réseau, elle est propagée à travers les différentes couches de neurones, de la couche d'entrée à la couche de sortie. À chaque couche, les données sont transformées par des opérations mathématiques, généralement des multiplications de matrices suivies de l'application d'une fonction d'activation.

Calcul de l'erreur : Une fois que les données ont été propagées à travers le réseau et que la sortie finale est générée, l'erreur entre la sortie prédite et la sortie réelle est calculée à l'aide d'une fonction de perte ou de coût. Cette fonction mesure à quel point les prédictions du réseau diffèrent des véritables valeurs cibles. Il existe plusieurs fonctions de calcul de perte, et elles sont classées selon le problème qu'on doit traiter :

1. Fonctions de perte de régression :
 - Perte d'erreur quadratique moyenne.
 - Perte d'erreur logarithmique quadratique moyenne.
 - Perte d'erreur absolue moyenne.
2. Fonctions de perte de classification binaire :
 - Entropie croisée binaire
 - Perte de charnière.
 - Perte de charnière carrée.
3. Fonctions de perte de classification multiclasse :
 - Perte d'entropie croisée multiclasse.
 - Perte d'entropie croisée multiclasse clarifiée.
 - Perte de divergence kullback leibler.

Rétropropagation (backpropagation) : L'erreur est ensuite propagée en arrière à travers le réseau à l'aide de l'algorithme de rétropropagation. Cela permet de calculer les gradients de l'erreur par rapport aux poids du réseau, ce qui indique comment les poids doivent être ajustés pour réduire l'erreur.

Mise à jour des poids : Les poids du réseau sont mis à jour en utilisant les gradients calculés lors de la rétropropagation, souvent à l'aide d'algorithmes d'optimisation tels que la descente de gradient stochastique (SGD) ou ses variantes, comme Adam ou RMSprop.

Itération : Les étapes de propagation avant, de calcul de l'erreur, de rétropropagation et de mise à jour des poids sont répétées de manière itérative sur de nombreuses itérations jusqu'à ce que le réseau atteigne un niveau de performance satisfaisant sur les données d'entraînement.

Évaluation : Une fois l'entraînement terminé, le réseau peut être évalué sur des données de test pour évaluer sa performance sur des données qu'il n'a pas vues pendant l'entraînement. Ce processus d'apprentissage profond permet au réseau de neurones d'ajuster ses poids pour minimiser l'erreur entre ses prédictions et les vraies valeurs cibles, ce qui lui permet d'apprendre à effectuer des tâches telles que la classification, la régression, la génération de données.

3.6 implementation d'un reseau de neurones

Pour fair l'apprentissage et l'implementation d'un reseau de neurones avec python a besoin plusieurs librairie de python qu'on va cite ci-dessous :

3.6.1 la partie apprentissage et manipulations des donnees

pour cette partie on aura besoin des ces librairies :

- **numpy** : elle permet de faire des calcul numerique et la gestion des tableaux de donnees et matrices,et elle possede la plupart des fonctions usuelles tel que logarithme,exponentielle.
- **pandas** : cette librairie est basee sur numpy et permet de manipuler facilement des donnees structurees(gerer les nouvelles colonnes,filtrer des donnees,calculer des metrique,etc).
- **Matplotlib** : on l'utilise pour la representation et la visualisation des donnees.
- **Seaborn** : est un outil de visualisation de donnees qui adaptee a pandas et qui vient souvent en complement a matplotlib,car elle est repute pour etre plus fonctionnelle et plus esthetiques.

3.6.2 La partie de creation du reseau de neurones

- **TensorFlow** : TensorFlow est l'une des bibliothèques les plus populaires pour le deep learning.Il offre une grande flexibilité pour la définition et l'entraînement de réseaux de neurones, ainsi que des outils avancés pour le déploiement de modèles.
- **PyTorch** : PyTorch est une autre bibliothèque populaire qui offre une interface plus dynamique et intuitive que TensorFlow. Elle est particulièrement appréciée pour la recherche en deep.
- **Keras** : Keras est une bibliothèque haut niveau qui peut être utilisée avec TensorFlow ou d'autres moteurs de backend comme Theano. Elle offre une API simple et intuitive pour la construction et l'entraînement de modèles de deep learning.

Conclusion

Dans ce chapitre,Nous avons donné un aperçu sur les réseaux de neurones artificiels. Nous avons défini ce qu'est un réseau de neurones, modélisé un neurone artificiel, et étudié leur structure et leur fonctionnement. Nous avons également examiné différents types de réseaux, comme les réseaux feedforward et récurrents, et abordé les concepts de l'apprentissage profond. En conclusion, ce chapitre nous a fourni une base solide pour comprendre les réseaux de neurones et leur utilisation dans des applications réelles. Dans le prochain chapitre, nous mettrons en pratique ces connaissances en présentant un réseau de neurones et en visualisant sa structure à l'aide de la bibliothèque NetworkX.

4

Application

Introduction

Dans ce chapitre, nous présentons un exemple d'approximation d'une courbe à l'aide d'un réseau de neurones. Cet exemple montre et détaille comment résoudre un problème de régression non linéaire avec TensorFlow.

Après avoir créé le réseau de neurones, nous allons le dessiner avec Networkx.

La suite de ce chapitre a été réalisée en ligne sur Google Colaboratory.

Sommaire

Introduction	30
4.1 Création du Réseau de Neurones	30
4.2 Visualisation du Réseau de Neurones	33
Conclusion	34

4.1 Création du Réseau de Neurones

4.1.1 Définition du problème

Le but de cet exemple est d'approximer une fonction non linéaire donnée par l'équation suivante :

$$Y = 0.1 * X * \cos(X)$$

Les points bleus représente un échantillon de notre jeu d'apprentissage, la ligne rouge est l'approximation faite par le réseau de neurones artificiel.

4.1.2 Explications

Commençons par importer les bibliothèques nécessaires :

- . **numpy** pour les tableaux et les matrices
- . **matplotlib** pour afficher les graphiques
- . **Tensorflow** pour les réseaux de neurones
- . **math** pour la fonction cosinus

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from tensorflow import keras
```

```
import tensorflow as tf
```

```
import math
```

Ensuite, créons le jeu de données pour l'apprentissage. **x_data** est composé de 1000 points, et un bruit normal est ajouté sur l'axe des ordonnées pour chaque point :

```
# Créer des données bruyantes
```

```
x_data = np.linspace(-10, 10, num=1000)
```

```
y_data = 0.1*x_data*np.cos(x_data) + 0.1*np.random.normal(size=1000)
```

```
print('Données créées avec succès')
```

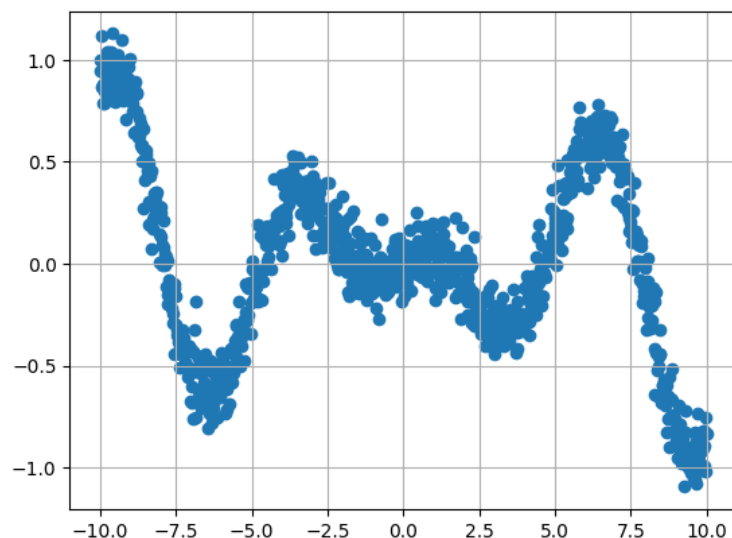
```
plt.scatter(x_data[ : :1], y_data[ : :1])
```

```
plt.grid()
```

```
plt.show()
```

sorti :

Données créées avec succès



Lorsque notre jeu de données est prêt, nous pouvons créer le réseau :

- . La première couche contient un seul neurone linéaire (pour l'entrée)
- . La seconde couche contient 10 unités de type RELU
- . La troisième couche contient 10 unités de type RELU
- . La dernière couche ne contient qu'une unité linéaire (pour la sortie)

Créer le modèle

```
model = keras.Sequential()
model.add(keras.layers.Dense(units = 1, activation = 'linear', input_shape=[1]))
model.add(keras.layers.Dense(units = 10, activation = 'relu'))
model.add(keras.layers.Dense(units = 10, activation = 'relu'))
model.add(keras.layers.Dense(units = 1, activation = 'linear'))
model.compile(loss='mse', optimizer="adam")
```

Afficher le modèle

```
model.summary()
```

Le modèle est compilé avec les paramètres d'optimisation suivants :

- . L'algorithme d'optimisation est Adam (**optimizer="adam"**).
- . Loss est la perte ou l'erreur calculée sur la base des moindres carrés (**loss='mse'**).

Une fois le modèle créé, entraînons notre réseau :

- . **x_data** est l'entrée
- . **y_data** est la sortie désirée
- . **epochs=5** signifie que notre réseau sera entraîné 5 fois sur notre jeu de données
- . **verbose=1** affiche la progression dans la console

entraînement

```
model.fit(x_data, y_data, epochs=700, verbose=1)
```

Lorsque l'apprentissage est terminé, nous pouvons prédire l'approximation :

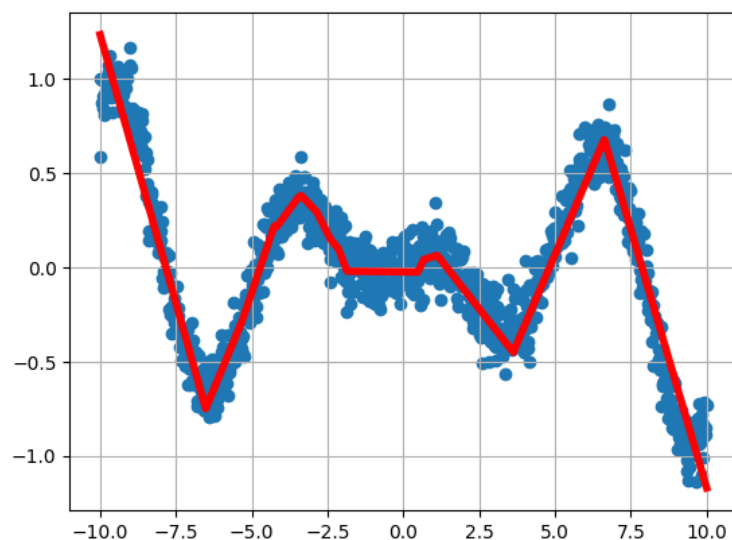
Calculer la sortie

```
y_predicted = model.predict(x_data)
```

Afficher le résultat

```
plt.scatter(x_data[:1], y_data[:1])
plt.plot(x_data, y_predicted, 'r', linewidth=4)
plt.grid()
plt.show()
```

Voici le résultat :



4.2 Visualisation du Réseau de Neurones

Grâce à l'utilisation des bibliothèques Networkx et Matplotlib, il est possible de représenter visuellement le réseau de neurones créé.

voici le code :

```
#importation les bibliothèques
import networkx as nx
import matplotlib.pyplot as plt

# Création d'un graphe vide
G = nx.MultiDiGraph()

# Ajout des couches
layers = ['entrée', 'caché1', 'caché2', 'sorti']

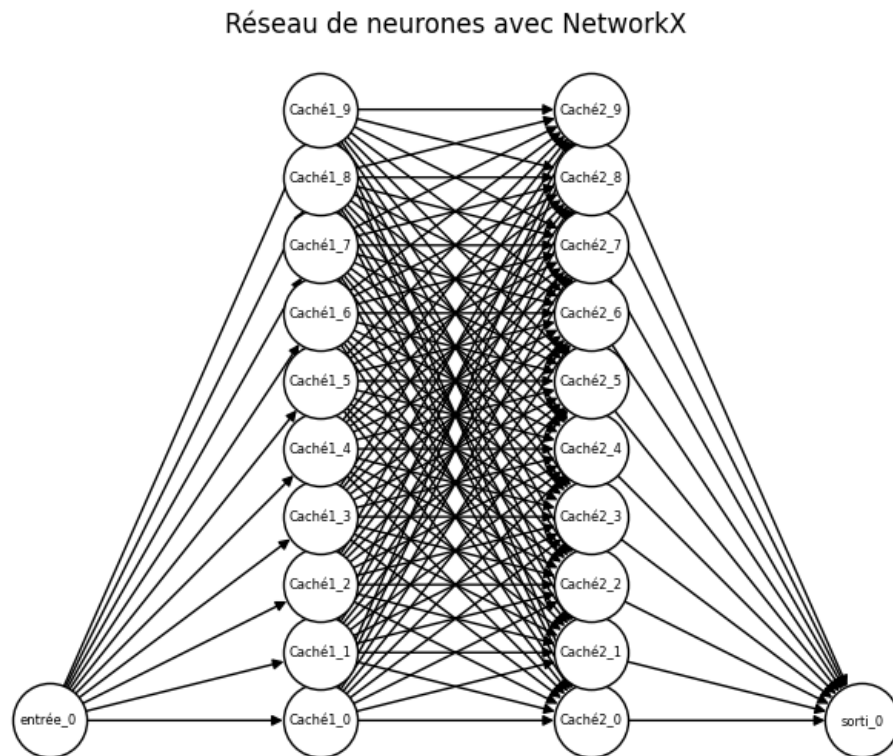
# Ajout des nœuds pour chaque couche
num_neurons = [1, 10, 10, 1] # Nombre de neurones dans chaque couche

# Décalage horizontal pour chaque couche
layer_shift = 50
for i, layer in enumerate(layers) :
    for j in range(num_neurons[i]) :
        node_label = layer + '_' + str(j)
        G.add_node(node_label, layer=layer, pos=(layer_shift * i, j))

# Ajout des connexions entre les couches
for i in range(len(layers) - 1) :
    current_layer = layers[i]
    next_layer = layers[i + 1]
    for j in range(num_neurons[i]) :
        for k in range(num_neurons[i + 1]) :
            current_node = current_layer + '_' + str(j)
            next_node = next_layer + '_' + str(k)
            G.add_edge(current_node, next_node)

# Dessin du graphe
pos = nx.get_node_attributes(G, 'pos')
nx.draw(G, pos, with_labels=True, node_size=1200, node_color='white', edgecolors='black', width=1,
font_size=6)
plt.title('Réseau de neurones avec NetworkX')
plt.show()
```

Voici le résultat :



Conclusion

Ce chapitre nous a donné l'occasion de passer de la théorie à la pratique en élaborant un réseau de neurones simple et en visualisant sa structure à l'aide de NetworkX. Cette expérience concrète nous a permis de mieux comprendre les concepts abordés dans les chapitres précédents.

Conclusion générale

Ce projet a permis d'explorer deux domaines clés des mathématiques appliquées : la théorie des graphes et les réseaux de neurones.

Dans la première partie du projet, nous avons étudié la théorie des graphes, découvrant les structures et les propriétés des graphes. L'utilisation de la bibliothèque NetworkX nous a permis de créer, manipuler et visualiser ces graphes, rendant ainsi les concepts théoriques plus applicables. Cette exploration nous a montré comment les graphes peuvent être utilisés pour modéliser des relations complexes dans divers domaines tels que les réseaux sociaux et les systèmes de transport.

La deuxième partie de ce projet a été consacrée aux réseaux de neurones, une technologie clé de l'intelligence artificielle et de l'apprentissage automatique. Nous avons commencé par définir ce qu'est un neurone artificiel et comment il est modélisé, avant d'examiner leur fonctionnement en détail.

Le projet s'est conclu par la mise en pratique des connaissances acquises dans les chapitres précédents. Nous avons présenté un réseau de neurones simple et visualisé sa structure en utilisant la bibliothèque NetworkX.

Ce projet a non seulement renforcé notre compréhension théorique des graphes et des réseaux de neurones, mais a également fourni des compétences pratiques en utilisant des outils comme NetworkX. Cette expérience nous prépare à aborder des problèmes complexes avec des approches innovantes et efficaces, et ouvre la voie à des applications avancées dans divers domaines de la science et de la technologie. Par cette intégration des connaissances théoriques et pratiques, nous sommes désormais mieux équipés pour contribuer aux avancées dans les domaines de la science des données et de l'intelligence artificielle.

Bibliographie

- [1] ABDI, H. *Les Réseaux de neurones*. Presses Universitaires de Grenoble, 1994.
- [2] ALAIN, M.-J., AND VALVERDE, I. L. *Introduction à la Théorie des Graphes*. Editions Universitaires, 2022.
- [3] COHEN, J. *Théorie des graphes et algorithmes*. oct 2006.
- [4] DATASCIENTEST. Networkx : Tout savoir. <https://datascientest.com/networkx-tout-savoir>.
- [5] DJERIRI, Y. *Les Réseaux de Neurones Artificiels*. september 2017.
- [6] HAGBERG, A. A., SWART, P., AND CHULT, D. S. Networkx. <https://networkx.org/>.
- [7] MOLKHIR, R., AND SIHAM, S. Réseaux de neurones pour les jeux de plateau. Master's thesis, Université Akli Mohand Oulhadj-Bouira, 2020.
- [8] MULLER, D. *Introduction à la théorie des graphes*. 2008.
- [9] PARIZEAU, M. *Réseaux de neurones GIF-21140 et GIF-64326*. Université Laval, 2004.
- [10] WEST, D. *Introduction to Graph Theory*, 2nd ed. Prentice Hall, 2000.